



# WEB班ワークショップ説明資料案2（初心者手取り足取り版）



## 今日のゴール

完成形のタスク管理アプリを段階的に再現して、CSS装飾とJavaScript動作の実装力を身につける



## 安心してください！

- コード量は合計約300行程度（HTML：70行、CSS：170行、JS：60行）
- 各段階で完成形のコードを見せるので、迷うことはありません
- ペアで協力しながら進めるので、一人で悩む必要はありません



## ワークショップの流れ

### Step 0: 完成形を体験しよう（10分）

 まずは触ってみよう！

完成形のタスク管理アプリを実際に操作してみてください。

操作してみること：

1.  チェックボックスをクリック
2.  タスクにマウスを乗せる（ホバー）
3.  進捗バーの変化を見る
4.  全部チェックして完了メッセージを確認



ペアで話し合おう（5分）

質問：「今操作したアプリで、どんな要素が見つかりましたか？」

話し合いのヒント：

- 画面上に見える要素は何個ぐらい？
- クリックできる場所はどこ？
- 自動で変わる部分はどこ？

期待する発見（後で確認します）：

- ヘッダー部分（タイトルと説明）
- 進捗表示部分（バーと数字）
- 6つのタスクチェックリスト
- マウスホバー時の動き
- 全完了時のポップアップメッセージ

## Step 1: HTMLで骨組みを作ろう（30分）

### まず完成形のHTMLを見てみよう

不安にならないよう、まずは完成形のHTMLコードを見てみましょう。

```
<!DOCTYPE html>
<html lang="ja">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>学習タスク管理</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <!-- ヘッダー -->
  <header class="main-header">
    <h1><img alt="book icon" data-bbox="198 488 218 500"/> 今日の学習タスク</h1>
    <p>目標: CSS装飾とJavaScript動きをマスターしよう!</p>
  </header>

  <!-- 進捗表示 -->
  <section class="progress-section">
    <h2><img alt="progress bar icon" data-bbox="198 598 218 610"/> 学習進捗</h2>
    <div class="progress-container">
      <div class="progress-bar">
        <div class="progress-fill" id="progressFill"></div>
      </div>
      <p class="progress-text" id="progressText">0 / 6 完了 (0%)</p>
    </div>
  </section>

  <!-- タスクリスト -->
  <section class="tasks-section">
    <h2><img alt="checkmark icon" data-bbox="198 798 218 810"/> タスクリスト</h2>

    <div class="task-item">
      <input type="checkbox" id="task1" class="task-checkbox">
      <label for="task1">HTML基本構造を作成</label>
    </div>

    <!-- 他のタスクも同様... -->
  </section>
```

```
<!-- 完了メッセージ -->
<div class="completion-message" id="completionMessage">
  <h2>🎉 おめでとうございます！</h2>
  <p>全てのタスクが完了しました！</p>
</div>

<script src="script.js"></script>
</body>
</html>
```

👁️ どうですか？ 思ったより単純だと思いませんか？

😓 どこから作り始めるか考えよう（10分）

**選択肢クイズ：** HTMLを書く順序として、どれが良いと思いますか？

A) 上から順番に（header → section → section...） B) 重要な部分から（タスクリスト → 進捗 → ヘッダー） C) 簡単な部分から（ヘッダー → タスクリスト → 進捗）

👉 ペアで話し合って、理由も考えてみてください。

▶ 💡 答えとその理由

💻 実装しよう（20分）

**今の目標：** 見た目は気にせず、機能する骨組みを作る

ステップ1-1: ヘッダー部分（5分）

```
<!DOCTYPE html>
<html lang="ja">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>学習タスク管理</title>
</head>
<body>
  <header class="main-header">
    <h1>📚 今日の学習タスク</h1>
    <p>目標: CSS装飾とJavaScript動きをマスターしよう！</p>
  </header>
</body>
</html>
```

**質問：** なぜ <div> ではなく <header> タグを使うのでしょうか？

**3段階ヒント：**

- **レベル1:** HTMLの意味を考えてみてください
- **レベル2:** <header> はページの頭部分を表します
- **レベル3:** セマンティック（意味的）なHTMLでSEOやアクセシビリティが向上します

## ステップ1-2: 進捗表示部分 (7分)

**注意:** 進捗バーは「外側div + 内側div」の2重構造です！

```
<section class="progress-section">
  <h2>📊 学習進捗</h2>
  <div class="progress-container">
    <div class="progress-bar">
      <div class="progress-fill" id="progressFill"></div>
    </div>
    <p class="progress-text" id="progressText">0 / 6 完了 (0%)</p>
  </div>
</section>
```

**質問:** なぜ進捗バーが2重divなのでしょう？

**選択肢:**

- A) 外側が枠、内側が実際のバー
- B) なんとなく2つ必要
- C) 1つだと動かない

👉 ペアで話し合ってください

▶ 💡 正解と説明

## ステップ1-3: タスクリスト部分 (8分)

```
<section class="tasks-section">
  <h2>✅ タスクリスト</h2>

  <div class="task-item">
    <input type="checkbox" id="task1" class="task-checkbox">
    <label for="task1">HTML基本構造を作成</label>
  </div>

  <div class="task-item">
    <input type="checkbox" id="task2" class="task-checkbox">
    <label for="task2">CSSで基本レイアウト</label>
  </div>

  <!-- 残り4つも同じパターンで作成してください -->
</section>

<!-- 完了メッセージ -->
<div class="completion-message" id="completionMessage">
  <h2>🎉 おめでとうございます！</h2>
  <p>全てのタスクが完了しました！</p>
</div>

<script src="script.js"></script>
```

### よくある質問への回答:

❓ 「labelタグって何？」 → チェックボックスとテキストを関連付けるタグ。 `for="task1"` と `id="task1"` で連携

❓ 「class名はどう決めればいい？」 → 後でCSSで使うので、内容がわかる名前（task-item, task-checkbox など）

❓ 「完了メッセージはなぜbodyの最後？」 → 最初は非表示で、JavaScriptで表示するため

## Step 2: CSSできれいに装飾しよう（35分）

### 📋 まず完成形のCSSを見てみよう

安心してください。CSSも段階的に説明します。

```
/* 基本設定 */
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}

body {
  font-family: "Hiragino Sans", "Yu Gothic", sans-serif;
  line-height: 1.6;
  color: #333;
  background: #f3f4f6;
  min-height: 100vh;
  padding: 20px;
}

/* ヘッダー */
.main-header {
  background: white;
  padding: 2rem;
  border-radius: 10px;
  text-align: center;
  margin-bottom: 2rem;
  box-shadow: 0 2px 10px rgba(0, 0, 0, 0.1);
}

/* セクション共通 */
section {
  background: white;
  padding: 2rem;
  border-radius: 10px;
  margin-bottom: 2rem;
  box-shadow: 0 2px 10px rgba(0, 0, 0, 0.1);
  border-left: 4px solid #4f46e5;
```

```
}

/* 進捗バー */
.progress-bar {
  width: 100%;
  height: 25px;
  background: #e5e7eb;
  border-radius: 12px;
  overflow: hidden;
  margin-bottom: 1rem;
}

.progress-fill {
  height: 100%;
  background: #10b981;
  border-radius: 12px;
  width: 0%;
  transition: width 0.5s ease;
}

/* タスクアイテム */
.task-item {
  display: flex;
  align-items: center;
  padding: 1rem;
  margin-bottom: 1rem;
  background: #f8fafc;
  border-radius: 8px;
  border: 2px solid #e5e7eb;
  cursor: pointer;
  transition: all 0.3s ease;
}

.task-item:hover {
  background: #f1f5f9;
  border-color: #4f46e5;
  transform: translateY(-2px);
  box-shadow: 0 4px 12px rgba(79, 70, 229, 0.2);
}
```

👁️ どうですか？ 意外と理解できそうではありませんか？

👁️ 現状と完成形の違いを見つけよう（5分）

👉 ペアで話し合い：今のHTMLと完成形を比べて、何が違いますか？

選択肢（複数選択可）：

- A) 背景色がない
- B) カード型の白い背景がない
- C) 余白・間隔が足りない
- D) 影や立体感がない

- E) フォントが違う

▶💡 正解

## 🍷 段階的に装飾していこう (25分)

### ステップ2-1: 全体の基本設定 (5分)

まず基礎を固めましょう:

```
* {  
  margin: 0;          /* 全要素の外側余白をリセット */  
  padding: 0;         /* 全要素の内側余白をリセット */  
  box-sizing: border-box; /* 重要！サイズ計算を簡単に */  
}  
  
body {  
  background: #f3f4f6; /* グレーの背景色 */  
  font-family: sans-serif; /* きれいなフォント */  
  padding: 20px;        /* 全体に余白 */  
}
```

質問: `box-sizing: border-box` って何ですか？

### 3段階ヒント:

- レベル1: 要素のサイズ計算方法を変更します
- レベル2: `border` や `padding` が `width` に含まれるようになります
- レベル3: 例: `width:100px + padding:10px = 合計100px` (`border-box`の場合)

📖 実際に書いて、変化を確認してみてください！

### ステップ2-2: カード型レイアウト (10分)

白いカードの効果を作りましょう:

```
.main-header, section {  
  background: white;          /* 白い背景 */  
  padding: 2rem;              /* 内側の余白 */  
  border-radius: 10px;        /* 角を丸く */  
  margin-bottom: 2rem;        /* 下の余白 */  
  box-shadow: 0 2px 10px rgba(0,0,0,0.1); /* 影で立体感 */  
}
```

詳しく理解しよう:

質問: `box-shadow: 0 2px 10px rgba(0,0,0,0.1)` の意味は？

選択肢クイズ:

- A) 水平0 垂直2px ぼかし10px 黒っぽい色
- B) 幅0 高さ2px 厚さ10px 透明な色
- C) よくわからない

👉 ペアで予想してから、実際に数値を変更して確認してみてください

実験してみよう：

```
/* 影を大きくしてみる */
box-shadow: 0 5px 20px rgba(0,0,0,0.3);

/* 影を右にずらしてみる */
box-shadow: 5px 2px 10px rgba(0,0,0,0.1);

/* 影の色を変えてみる */
box-shadow: 0 2px 10px rgba(255,0,0,0.3);
```

ステップ2-3: 進捗バーとタスクアイテム (10分)

進捗バーのスタイル：

```
.progress-bar {
  width: 100%;
  height: 25px;
  background: #e5e7eb;    /* 薄いグレー（枠の色） */
  border-radius: 12px;
  overflow: hidden;      /* 重要！内側がはみ出さないように */
}

.progress-fill {
  height: 100%;
  background: #10b981;    /* 緑色（進捗の色） */
  border-radius: 12px;
  width: 0%;              /* 最初は0%（JavaScriptで変更） */
  transition: width 0.5s ease; /* なめらかに変化 */
}
```

質問：なぜ `width: 0%` なのですか？

3段階ヒント：

- ・ **レベル1**: 最初の状態を考えてみてください
- ・ **レベル2**: タスクが0個完了の時は何%でしょうか？
- ・ **レベル3**: JavaScriptでチェック状況に応じて `width` を変更します

タスクアイテムのスタイル：

```
.task-item {
  display: flex;          /* 横並びレイアウト */
  align-items: center;    /* 縦方向の中央揃え */
  padding: 1rem;
  margin-bottom: 1rem;
  background: #f8fafc;
  border-radius: 8px;
  border: 2px solid #e5e7eb;
```



```
cursor: pointer;          /* マウスカースルを手の形に */  
}
```

### ✅ 完成形チェック (5分)

👉 **ペアで確認:** 静的な見た目が完成形と同じになりましたか？

チェックポイント:

- ✅ 背景がグレーになった
- ✅ 白いカードっぽくなった
- ✅ 進捗バーが表示された
- ✅ タスクアイテムがきれいに並んだ

⚠️ まだホバーエフェクトがないのは正常です！次のステップで追加します。

## Step 3: アニメーションで動きを付けよう (25分)

📄 **まず完成形のアニメーション部分を見てみよう**

```
/* ホバーエフェクト */  
.task-item {  
  transition: all 0.3s ease; /* なめらか変化の準備 */  
}  
  
.task-item:hover {  
  background: #f1f5f9;      /* 背景色を少し変更 */  
  border-color: #4f46e5;    /* ボーダー色を青に */  
  transform: translateY(-2px); /* 2px上に移動 */  
  box-shadow: 0 4px 12px rgba(79, 70, 229, 0.2); /* 影を大きく */  
}  
  
/* 完了状態のスタイル */  
.task-item.completed {  
  background: #ecfdf5;      /* 薄い緑背景 */  
  border-color: #10b981;    /* 緑のボーダー */  
}  
  
.task-item.completed label {  
  text-decoration: line-through; /* 取り消し線 */  
  color: #6b7280;           /* グレーの文字色 */  
}
```

### 🔍 どんな動きがあるか観察しよう (5分)

👉 **ペアで話し合い:** 完成形でマウスを動かした時、何が起こっていましたか？

選択肢 (複数選択可):

- A) 背景色が変わる

- B) 少し浮き上がる
- C) 影が大きくなる
- D) 境界線の色が変わる
- E) 回転する

▶ 💡 正解

### ✨ ホバーエフェクトを作ろう (15分)

#### ステップ3-1: 基本のホバー効果 (5分)

まずは簡単な背景色変化から：

```
.task-item:hover {  
  background: #f1f5f9;    /* 少し青っぽい背景に */  
}
```

📖 実際に書いて、マウスを乗せてみてください！

質問：色の変化が急すぎませんか？

解決法： `transition` を追加しましょう

```
.task-item {  
  transition: all 0.3s ease; /* これを追加！ */  
}
```

#### ステップ3-2: 浮き上がり効果 (5分)

立体感を作るテクニック：

```
.task-item:hover {  
  background: #f1f5f9;  
  transform: translateY(-2px); /* 2px上に移動 */  
  box-shadow: 0 4px 12px rgba(79, 70, 229, 0.2); /* 影を大きく */  
}
```

実験してみよう：

- `translateY(-2px)` を `-5px` や `-10px` に変えてみてください
- 影の数値も変更してみてください

質問：なぜ `translateY(-2px)` で浮き上がって見えるのですか？

3段階ヒント：

- レベル1: Y軸は縦方向の移動です
- レベル2: マイナスの値は上方向への移動です
- レベル3: 影と組み合わせることで、浮いているように錯覚します

#### ステップ3-3: 仕上げの装飾 (5分)

最終的なホバー効果：

```
.task-item:hover {  
  background: #f1f5f9;  
  border-color: #4f46e5;    /* ボーダー色も変更 */  
  transform: translateY(-2px);  
  box-shadow: 0 4px 12px rgba(79, 70, 229, 0.2);  
}
```

### 動作確認 (5分)

 ペアでチェック: 完成形と同じアニメーションになっていますか?

確認ポイント:

- ☒ マウスを乗せると浮き上がる
- ☒ なめらかに変化する
- ☒ 影が大きくなる
- ☒ 境界線の色が変わる

## Step 4: JavaScriptで動的機能を作ろう (20分)

 まず完成形のJavaScriptを見てみよう

```
document.addEventListener("DOMContentLoaded", function () {  
  // 要素を取得  
  const taskCheckboxes = document.querySelectorAll(".task-checkbox");  
  const progressFill = document.getElementById("progressFill");  
  const progressText = document.getElementById("progressText");  
  const completionMessage = document.getElementById("completionMessage");  
  
  // 各チェックボックスにイベントを追加  
  taskCheckboxes.forEach((checkbox) => {  
    checkbox.addEventListener("change", function () {  
      handleTaskChange(this);  
    });  
  });  
  
  // タスクの状態が変更された時の処理  
  function handleTaskChange(checkbox) {  
    const taskItem = checkbox.closest(".task-item");  
  
    if (checkbox.checked) {  
      taskItem.classList.add("completed");  
    } else {  
      taskItem.classList.remove("completed");  
    }  
  
    updateProgress();  
  }  
}
```

```
// 進捗バーの更新
function updateProgress() {
    const totalTasks = taskCheckboxes.length;
    const completedTasks = document.querySelectorAll(".task-checkbox:checked").length;
    const percentage = Math.round((completedTasks / totalTasks) * 100);

    progressFill.style.width = percentage + "%";
    progressText.textContent = `${completedTasks} / ${totalTasks} 完了 (${percentage}%)`;

    if (completedTasks === totalTasks) {
        completionMessage.style.display = "block";
    } else {
        completionMessage.style.display = "none";
    }
}

updateProgress();
});
```

👁️ どうですか？ 思ったより読めそうではありませんか？

🧠 動的機能を整理しよう（5分）

👉 ペアで話し合い：JavaScriptで実現したい機能は何ですか？

選択肢：

- A) チェック → タスクに取り消し線
- B) チェック → 進捗バー更新
- C) 全完了 → メッセージ表示
- D) 音楽を再生
- E) 背景色を変更

▶ 💡 正解

⚙️ 段階的に実装しよう（15分）

ステップ4-1: 要素を取得（3分）

HTMLの要素を JavaScript で操作するための準備：

```
document.addEventListener("DOMContentLoaded", function () {
    // 要素を取得
    const taskCheckboxes = document.querySelectorAll(".task-checkbox");
    const progressFill = document.getElementById("progressFill");
    const progressText = document.getElementById("progressText");
    const completionMessage = document.getElementById("completionMessage");
});
```

詳しく理解しよう：

質問: `querySelectorAll` と `getElementById` の違いは？

選択肢:

- A) `querySelectorAll` は複数、`getElementById` は1つ
- B) `querySelectorAll` は新しい、`getElementById` は古い
- C) 違いはない

👉 ペアで話し合ってください

▶ 💡 正解と説明

ステップ4-2: イベント処理 (7分)

チェックボックスがクリックされた時の処理:

```
// 各チェックボックスにイベントを追加
taskCheckboxes.forEach((checkbox) => {
  checkbox.addEventListener("change", function () {
    console.log("チェックボックスがクリックされました!");
    // ここに処理を書く
  });
});
```

🖥️ まずは `console.log` で動作確認してみてください！ (F12 → Console タブで確認)

次に実際の処理を追加:

```
taskCheckboxes.forEach((checkbox) => {
  checkbox.addEventListener("change", function () {
    const taskItem = checkbox.closest(".task-item");

    if (checkbox.checked) {
      // チェックされた時
      taskItem.classList.add("completed");
    } else {
      // チェック外された時
      taskItem.classList.remove("completed");
    }
  });
});
```

質問: `closest(".task-item")` って何ですか？

3段階ヒント:

- レベル1: 親要素を探すメソッドです
- レベル2: チェックボックスの親の `task-item` を見つけます
- レベル3: `<div class="task-item"><input></div>` の `div` を取得

ステップ4-3: 進捗計算・表示 (5分)

進捗バーを更新する処理:

```
function updateProgress() {
  const totalTasks = taskCheckboxes.length; // 全タスク数
  const completedTasks = document.querySelectorAll(".task-checkbox:checked").length; // 完了数
  const percentage = Math.round((completedTasks / totalTasks) * 100); // パーセント

  // 進捗バー更新
  progressFill.style.width = percentage + "%";
  progressText.textContent = `${completedTasks} / ${totalTasks} 完了 (${percentage}%)`;

  // 全完了判定
  if (completedTasks === totalTasks) {
    completionMessage.style.display = "block";
  } else {
    completionMessage.style.display = "none";
  }
}

// イベント処理の最後に追加
updateProgress();
```

計算を理解しよう：

例：6個中2個完了の場合

- `totalTasks = 6`
- `completedTasks = 2`
- `percentage = Math.round((2 / 6) * 100) = 33%`



## 完成！おめでとうございます！

### できるようになったこと

- ☒ HTMLで意味的な構造を設計
- ☒ CSSでカード型レイアウトと影効果
- ☒ ホバーエフェクトの実装
- ☒ JavaScriptでDOM操作とイベント処理
- ☒ 進捗計算とリアルタイム更新



### ペアで振り返ろう

1. 今日一番難しかった部分は？
2. 今日一番面白かった部分は？
3. 次に挑戦してみたい機能は？

# SOS 困った時のサポート

## 段階的ヒントシステム

### レベル1: 考える方向を示す

- 「〇〇について考えてみてください」
- 「完成形と比べて何が違いますか？」

### レベル2: 選択肢を提示

- 「A, B, C のどれが良いと思いますか？」
- 「〇〇という方法はどうでしょう？」

### レベル3: 具体的な解決法

- 「この部分のコードを確認してみましょう」
- 「〇〇をこのように書いてみてください」

## よくあるエラーと解決法

### HTML関連

✗ 「要素が表示されない」 → タグの閉じ忘れがないか確認 → class名やid名のスペルミスがないか確認

### CSS関連

✗ 「スタイルが効かない」 → CSSのセレクタ名とHTMLのclass/id名が一致しているか確認 → ブラウザのキャッシュをクリア (Ctrl + F5)

✗ 「ホバーエフェクトが急に変わる」 → `transition` プロパティを追加

### JavaScript関連

✗ 「動作しない」 → F12 → Console でエラーメッセージを確認 → HTML要素のid/class名とJavaScriptが一致しているか確認

✗ 「要素が取得できない」 → `DOMContentLoaded` イベント内に行っているか確認 → HTML要素が存在するか確認

## デバッグのコツ

1. `console.log()`を活用: 変数の中身を確認
2. 段階的に確認: 一つずつ動作確認
3. ペアと相談: 二人で見ると気づくことが多い

## 質問の仕方

✖ 「動きません」    ✔ 「〇〇を実現したいのですが、△△の部分でエラーが出ます」

✖ 「わかりません」

✔ 「〇〇は理解できましたが、△△の理由がわかりません」

---

Happy Coding! 🚀 次回はさらに高度な機能に挑戦しましょう！